Session 3 Laboratory Stakeholders and Requirements

Stakeholders and Requirements

Worked Example and Student Exercises

Read the complete worked example first. Then complete the guided questions and the independent student scenario. The purpose is to learn a repeatable analysis process, not to copy phrases from the example.

Laboratory overview

This laboratory develops the requirements-analysis skills introduced in the lecture. The first section demonstrates a complete worked example. The second section provides short guided exercises. The final section gives a new scenario for independent analysis.

Learning outcomes

- Distinguish a business problem from a proposed solution or feature.
- Write a concise business-problem statement.
- Define system purpose and initial scope.
- Identify direct, indirect and external stakeholders.
- Write clear functional requirements.
- Rewrite vague quality statements as measurable non-functional requirements.
- Distinguish requirements, business rules, assumptions, constraints and risks.
- Prioritise requirements using MoSCoW.

- Identify questions that require stakeholder clarification.

Laboratory outputs

- Completed guided-analysis questions.
- One independent business-problem statement.
- One purpose and scope statement.
- A stakeholder-analysis table.
- Eight functional requirements.
- Four measurable non-functional requirements.
- Business rules, assumptions, constraints and risks.
- A requirement-quality review.

Concept guide and working method

Use this section as a reference while completing the worked example and exercises. The concepts are deliberately explained before they are applied so that the laboratory develops a repeatable method rather than a collection of isolated answers.

A disciplined requirements-analysis workflow

Stage	What to do	Why it matters
Read	Read the whole scenario once without	Prevents early details from hiding the overall problem.
Annotate	Mark current problems, stakeholders, rules, data, constraints and	Creates evidence for later decisions.
Classify	Separate problems, needs, requirements, assumptions,	Prevents categories from being confused.
Specify	Write uniquely identified, atomic and verifiable	Creates statements that can be reviewed and tested.
Prioritise	Apply MoSCoW using business value,	Makes release choices explicit.

Review	Check clarity, necessity, consistency, feasibility, traceability and solution	Improves quality before design begins.

Business problem, desired outcome and feature

These three ideas are related, but they are not interchangeable. A business problem describes an undesirable current condition. A desired outcome describes the improvement the organisation wants. A feature is one possible system behaviour that may contribute to the outcome.

Type	Question	Example
Business problem	What is wrong or difficult now?	Members must contact staff to discover availability.
Desired outcome	What improvement is needed?	Members obtain accurate availability information without
Possible feature	What system behaviour might	The system displays the current status of each facility

Analysis should begin with the problem and desired outcome. A proposed feature should not be accepted automatically until its value, scope and consequences are understood.

Stakeholder versus actor

A stakeholder is any person, group, organisation or external body that affects the proposed system, is affected by its outcomes, supplies information, receives information, governs policy, funds the work or supports operation. An actor is an external role or system that directly interacts with the system boundary. An individual or role may be both, but the terms are not synonyms.

Stakeholder category	Guiding question	Typical example
Primary	Who seeks value from the	Member or participant

Operational	Who performs day-to-day	Reception staff or tutor
Management	Who approves policy or monitors performance?	Centre manager
Technical/	Who maintains access or	System administrator
External/ regulatory	Who imposes reporting or compliance obligations?	Regulator or local authority
Indirect	Who is affected without direct interaction?	Parent, community group or senior

What is a requirement specification?

A requirement specification is a structured record of the agreed capabilities, qualities, rules and constraints for a proposed system. It provides a common reference for stakeholders, analysts, designers, developers and testers. The specification may include textual requirements, models, rules, rationale, priorities, sources and verification information.

A useful requirement record

Field	**Purpose**
Identifier	Provides a stable reference such as FR01 or
Statement	Describes the required capability, quality or
Source	Records the stakeholder, policy or evidence
Rationale	Explains why the requirement is needed.
Priority	Supports release planning and negotiation.
Verification	Explains how satisfaction will be demonstrated.
Dependencies	Records related requirements, rules or external
Status	Shows whether the requirement is proposed,

The laboratory uses a simplified specification. The aim is to practise disciplined requirement writing and traceability, not to reproduce a large industrial standard.

Functional requirements

A functional requirement describes an observable service, capability, calculation, validation or response that the proposed system must provide. A strong functional requirement is atomic, clear, solution-neutral and verifiable.

Weak	Why it is weak	Improved requirement
Staff can manage	“Manage” combines several unspecified	FR01: Authorised staff shall create, amend and
Use a cloud database.	Premature implementation	FR02: The system shall retain active and historical
Participants get notifications.	Event, recipient and result are unclear.	FR03: The system shall notify registered participants when a
Search should be fast.	Combines a function with an undefined quality.	FR04: The system shall search activities by title, date and tutor. Add a

Recommended requirement pattern

FR<number>: The system shall <observable behaviour> [for <actor or condition>].

Non-functional requirements

A non-functional requirement describes a system quality, operating condition or constraint. It should normally include a measurable target or a condition that can be verified. Words such as fast, secure, reliable and user-friendly are starting points for discussion, not complete requirements.

Quality	Question to ask	Example measure
Performance	How quickly or how much?	95% of searches return within 3 seconds under an

Availabilit y	When must the service operate?	Available during published opening hours, excluding
Security	Who may access which function or information?	Only authorised staff may view emergency contact
Usability	Can a defined user complete a defined	A trained staff member completes a standard
Reliability	What failure or loss is acceptable?	Confirmed registrations survive a single application
Scalability	What future volume must be supported?	Support at least 5,000 active participant records.
Maintaina bility	How should controlled change be made?	Capacity rules are configurable without
Accessibil ity	Which access need or standard applies?	All participant activities support keyboard-only

Verification methods

A requirement is verifiable when evidence can show whether it has been satisfied. Different requirements may need different forms of evidence.

Method	**Meaning**	**Example**
Test	Execute the system with defined inputs and	Measure search response time.
Demonstr ation	Show the capability in a controlled scenario.	Register a participant and cancel a booking.
Inspection	Review an artefact, configuration or record.	Inspect role permissions or retention settings.
Analysis	Use calculation, modelling or reasoned	Analyse expected capacity under forecast load.

Business rules, assumptions, constraints, risks and future enhancements

Category	Meaning	How to manage it
Business rule	A policy or fact of the organisation or domain.	Trace the rule to requirements and
Assumption	Something believed true but not yet confirmed.	Record the owner and verify it.
Constraint	A restriction on the project or solution.	Treat it as a boundary for planning and design.
Risk	An uncertain event or condition that may	Assess likelihood, impact and response.
Future enhanceme	A possible later capability outside	Record it without allowing it to expand the current

MoSCoW prioritisation

MoSCoW is a negotiation technique for distinguishing essential scope from valuable but deferrable scope. The fourth category means “Won’t Have This Time”, not “never”. Priorities should reflect business purpose, dependency, risk and the availability of a temporary workaround.

Priority	Meaning	Decision test	Common
Must Have	Essential for the release to achieve	Would the release fail or be unusable	Calling an important
Should Have	Important and expected, but a	Could the release operate for a limited	Treating Should as
Could Have	Desirable if time and resources remain.	Does it add useful value without defining the core	Allowing Could items to displace
Won’t Have	Explicitly excluded from the current	Should it be reconsidered in a	Interpreting Won’t as

How to justify a priority

- State the business problem or stakeholder goal supported.
- Identify dependencies on other requirements.

- Explain operational, legal, privacy or safety risk.
- Identify any current workaround and its cost.
- State the consequence of deferring the requirement.

If every requirement is labelled Must Have, no meaningful prioritisation has occurred. A viable set of Must requirements should define the smallest release that still achieves the agreed purpose.

Characteristics of a strong requirement

Quality	**Review question**
Necessary	Does it support a stated business problem, rule
Atomic	Does it express one principal idea?
Clear	Do key terms have one reasonable
Consistent	Does it avoid conflict with another requirement
Feasible	Is it achievable within known constraints?
Verifiable	Can evidence show whether it has been
Traceable	Is its source and rationale recorded?
Solution-	Does it avoid an unnecessary implementation
Prioritised	Is the release importance justified?

Ambiguous words to avoid or define

Word or	**Why it is weak**	**Better approach**
manage	May mean create, amend, delete, approve	List the required actions.
fast	No operation, load or threshold is stated.	Specify operation, conditions and maximum
secure	No actor, asset, threat or control is identified.	State who may access which information or
user-friendly	No user group, task or measure is defined.	Specify a user, task and success measure.
appropriate	Depends on unstated	Identify the rule or

etc.	Hides incomplete scope.	List all included items or reference a controlled list.
modern	Subjective and technology-led.	State the required outcome or compatibility.
as soon as possible	No agreed time limit.	Specify the event and maximum elapsed time.

Part A: Fully worked example

Scenario: Riverside Community Sports Centre

Riverside Community Sports Centre currently manages memberships, facility bookings and class registrations using paper forms, spreadsheets and email. Booking requests arrive by telephone, email and at the reception desk. Staff sometimes receive conflicting requests for the same facility and time. Members must contact reception to check availability, and class waiting lists are maintained manually. Management cannot easily determine facility usage, class attendance or membership activity. The centre wishes to improve the accuracy and accessibility of this information.

Step 1: Extract facts about the current situation

Evidence from the scenario	**What the evidence tells us**
Memberships, bookings and classes use paper,	Information is distributed across several records and
Requests arrive by telephone, email and at reception.	There is no single, reliable booking process.
Conflicting requests occur.	Availability and overlap are not controlled consistently.

Members contact reception to	Routine information is not
Waiting lists are maintained	Queue order and updates
Management cannot easily obtain activity information.	Reporting information is fragmented or incomplete.

Step 2: Separate symptoms, problems and consequences

Observation or	**Underlying problem**	**Likely consequence**
Conflicting booking requests	No reliable central view of availability	Double booking and corrective work
Telephone availability enquiries	Members cannot access current	Routine enquiries consume staff time
Manual waiting lists	Queue order is not consistently	Members may be treated
Difficult reporting	Data is fragmented	Management decisions rely on

A symptom is what is observed. The underlying problem explains why the issue exists. The consequence explains the organisational effect.

Step 3: Worked business-problem statement

Riverside Community Sports Centre uses paper forms, spreadsheets and email to manage memberships, facility bookings and class registrations. Information is fragmented across several records and communication channels. Staff cannot reliably prevent conflicting facility bookings, members must contact reception to check availability, and class waiting lists require manual maintenance. Management also has difficulty producing accurate information about facility use, class attendance and membership activity. These issues increase administrative effort, reduce confidence in the centre's records and may result in inconsistent service for members.

Why this is a strong problem statement

- It identifies the organisation and current process.
- It explains the principal difficulties.
- It identifies affected groups.
- It describes operational consequences.
- It does not prescribe a database, application framework or screen design.

Step 4: Worked system-purpose statement

A purpose statement summarises what the proposed system should support. It stays at outcome level and avoids premature technology choices.

The proposed Sports Centre Management System will support membership records, facility availability, facility bookings, class registration, waiting-list management and basic operational reporting.

Step 5: Worked scope

In scope for the initial release	**Out of scope for the initial**
Member records	Staff payroll
Facility availability	Equipment purchasing
Facility bookings and	Supplier management
Class registration	Building-access control
Class waiting lists	Fitness-tracker integration
Attendance recording	Native mobile application
Basic usage reports	Full public website redesign

Out of scope does not mean unimportant. It states that the activity is not included in the current boundary.

Step 6: Worked stakeholder analysis

Stakehol	Goals	Concerns	Influen	Expected
Member	Find and book	Availability, privacy,	Medium	View availability, book and cancel
Reception staff	Process requests	Conflicts and inaccurate	High	Maintain members and
Class instructor	Access accurate	Capacity and attendance	Medium	View class lists; record
Centre manager	Monitor activity and	Reliable reports and	High	View and export reports
System administra	Maintain secure,	Security, availability	High	Manage access and system
Data-protection	Ensure appropriate	Privacy, retention and	High	Review requirements and

Stakeholder observations

- Not every stakeholder is a daily user.
- Influential stakeholders may set policy or reject the solution.
- Stakeholder goals can conflict. For example, management may want extensive reporting while privacy stakeholders may seek minimal retention.
- Stakeholders are sources of requirements, rules, constraints and clarification questions.

Step 7: Worked functional requirements

A useful starting pattern is:

<ID>: The system shall <observable capability> <under any important condition>.

ID	Functional requirement	Priority	Source
FR01	The system shall allow authorised reception staff to register a new	Must	Reception staff
FR02	The system shall allow an active member to view available facilities for a	Must	Member

FR03	The system shall prevent two confirmed bookings from occupying	Must	Centre manager
FR04	The system shall allow an active member to book an available facility.	Must	Member
FR05	The system shall record the member, facility, start time, end time and status	Must	Reception staff
FR06	The system shall allow a member to cancel a future booking subject to the	Should	Member
FR07	The system shall place class-registration requests on a waiting list	Must	Instructor
FR08	The system shall preserve the order in which waiting-list requests were	Must	Centre manager
FR09	The system shall allow authorised staff to record class attendance.	Should	Instructor
FR10	The system shall allow the manager to generate a facility-usage report for a	Could	Centre manager

What makes these requirements useful?

- Each requirement has a unique identifier.
- Each requirement describes observable behaviour.
- Important actors or conditions are identified.
- The wording avoids specifying implementation technology.
- Priority and source support negotiation and traceability.

Step 8: Worked non-functional requirements

Weak	**Improved requirement**	**Quality**
The system should be fast.	NFR01: Facility-availability results shall be displayed within three seconds for at least 95% of requests under normal	Performance
The system should be secure.	NFR02: Membership and booking-administration functions shall be accessible only to authenticated users	Security

The system should always be available.	NFR03: The system shall be available during published opening hours, excluding maintenance notified to staff	Availability
The system must protect member	NFR04: A member shall be able to view only the membership and booking records associated with that	Privacy
The system should be easy	NFR05: All member booking activities shall support keyboard-only	Accessibility

Measurability checklist

- Is there an observable result?
- Is a threshold, proportion or condition stated where appropriate?
- Is the target feasible and justified?
- Could a tester determine whether the requirement is satisfied?

Step 9: Rules, assumptions, constraints and risks

Type	Worked example	Explanation
Business rule	A facility booking must be at least 30 minutes long.	An organisational or domain policy
Business rule	A class may not exceed its approved capacity.	A rule that applies regardless of
Business rule	A suspended member may not make a new booking.	An eligibility rule
Assumption	Every current member has a unique membership number.	Believed true, but must be verified
Assumptio	Reception staff have supported	Requires verification
Constraint	The initial solution must operate through a web browser.	Restricts the solution

Constraint	Existing membership records must be imported.	Restricts transition and implementation
Risk	Existing spreadsheets may contain duplicates.	An uncertain condition that may

Step 10: MoSCoW priority

Priority	Meaning	Worked interpretation
Must	Essential to achieve the release purpose	Prevent conflicting confirmed bookings
Should	Important, but a temporary workaround	Allow online cancellation
Could	Desirable if time and	Advanced usage charts
Won’t this time	Explicitly deferred from the current release	Native mobile application

Part B: Guided practice

Guided exercise 1: Fragmented event registration (8 minutes)

Purpose: Move from an observation to a requirement and a stakeholder question.

Prompt: Event registrations are collected through several unrelated online forms.

Analysis question	Your answer
What is the current	
What is the underlying	
Who is affected?	

What consequence may	
Write one possible	
Write one stakeholder clarification question.	

Guided exercise 2: “We need a dashboard” (8 minutes)

Purpose: Separate stakeholder need from a proposed interface solution.

A coordinator states: “We need a dashboard.” Answer the following:

Why is “dashboard” a proposed solution rather than a complete requirement?

_____
_____

What information might the coordinator actually need?

_____
_____

Write two separate functional requirements that could satisfy the underlying need.

_____
_____

What questions would you ask before agreeing to the dashboard?

_____
_____

Guided exercise 3: “The system should support lots of users” (8 minutes)

Purpose: Transform a vague quality statement into a measurable requirement.

What does “lots” mean in this context?

_____

_____

Which operations matter under load?

_____

_____

What response-time or throughput threshold might need agreement?

_____

_____

Under what conditions should the target apply?

_____

_____

Write one measurable performance requirement.

_____

_____

Part C: Student exercises

Scenario: Harbour Arts and Workshop Centre

Harbour Arts and Workshop Centre runs evening courses, weekend workshops and community exhibitions. Course and

workshop registrations are currently received by email, telephone and paper forms. Participant details are copied into separate spreadsheets by different staff members. Staff cannot always determine whether a workshop has reached capacity, and waiting lists are managed through email folders. Participants sometimes receive late or inconsistent information about cancellations and schedule changes. Tutors receive attendance lists in different formats, and the centre manager finds it difficult to produce reliable information about enrolment, attendance and room use. Some activities require an age restriction or evidence of prior experience. The centre also needs to protect participant contact details and emergency information.

Task 1: Classify statements (8 minutes)

Purpose: Distinguish problem evidence from solution ideas, rules, assumptions and constraints.

Statement	Classificat	Reason
Participant details are copied into		
Participants should receive		
A workshop must not exceed the approved room capacity.		
The initial service must be accessible through a web		
Every tutor has access to email.		
The centre should use a cloud		
Some activities require participants to be at least 16		
Existing participant records may contain duplicate contact details.		

Task 2: Write the business-problem statement (12 minutes)

Purpose: Explain the current situation, difficulties and consequences without prescribing the technical solution.

Write 150–200 words. Include organisational context, affected stakeholders, principal difficulties and operational consequences.

_____

_____

Task 3: Define purpose and scope (10 minutes)

Write a one-sentence purpose statement:

_____

_____

In scope	Out of scope for the initial

Task 4: Stakeholder analysis (12 minutes)

Include at least one direct user, one managerial stakeholder, one technical stakeholder and one governance or external stakeholder.

Stakeholder	Goals	Concerns	Influence	Expected interaction

Task 5: Functional requirements (15 minutes)

Write eight requirements. Use unique identifiers, “shall” language, priorities and sources.

ID	Functional requirement	MoSC	Source /
FR0			
FR0			
FR0			
FR0			
FR0			
FR0			
FR0			
FR0			

Task 6: Non-functional requirements (12 minutes)

Write four measurable requirements. Cover at least three different quality categories.

ID	Quality	Measurable requirement	Rationale
NFR			
NFR			
NFR			
NFR			

Task 7: Rules, assumptions, constraints and risks (10 minutes)

Category	Statement

Business rule	
Business rule	
Business rule	
Assumption	
Assumption	
Constraint	
Constraint	
Risk	
Risk	

Task 8: Stakeholder clarification questions (8 minutes)

1. _____
   _____
2. _____
   _____
3. _____
   _____
4. _____
   _____
5. _____
   _____

Task 9: Requirement quality review (10 minutes)

Review question	Yes / No	Required improvemen
Each functional requirement contains one		
The wording has one reasonable		
The requirement is within scope.		
The requirement avoids unnecessary		
The requirement can be verified by demonstration, test, inspection or		
The source or stakeholder is recorded.		
The priority reflects a genuine delivery		
No requirement conflicts with a stated		
Non-functional requirements include		
Unresolved information is recorded as a question rather than invented.		

Exit reflection

Which requirement was hardest to make verifiable, and why?

_____
_____

Which stakeholder was least obvious?

_____
_____

Which statement initially confused a problem with a solution?

_____
_____

What additional information is needed before use-case modelling begins?

_____

_____

Part D: Review, extension and self-assessment

Partner review record

Exchange one functional and one non-functional requirement with a partner. Review the requirement rather than the person who wrote it.

Requirement	Issue identified	Suggested	Accepted?

Rewrite challenge

Rewrite each vague statement. Separate functional and non-functional concerns where necessary.

- The system should be secure.

_____

_____

- Users can manage their activities.

_____

_____

- Search should be fast and easy.

_____

_____

- The system will use a modern cloud database.

- Notifications should be sent promptly.

Requirement traceability exercise

Link four business problems to stakeholder goals and requirements. Mark any coverage gap.

Business	Stakeholder goal	Requireme	Coverage

Hidden solution assumptions

Review the requirements and highlight any statement that assumes a technology, deployment style, database, user-interface design or external product that has not been agreed. Rewrite the requirement in solution-neutral language.

Original statement	Hidden solution	Rewritten

Questions for stakeholders

Do not invent missing rules. Record the question, explain why the answer matters and identify the affected requirement.

Question	Why the answer matters	Affected requireme

Expected laboratory evidence

- A business-problem analysis supported by scenario evidence.
- A purpose statement and explicit initial scope.
- A stakeholder analysis containing direct, indirect, managerial, technical and governance perspectives.
- Eight functional requirements with IDs, sources and justified priorities.
- Four measurable non-functional requirements with proposed verification methods.
- A documented list of rules, assumptions, constraints and risks.
- Five stakeholder clarification questions.
- A completed quality review and partner feedback record.

Self-assessment

Statement	Not yet	Developing	Confident
I can distinguish a business problem, desired outcome and feature.	☐	☐	☐
I can distinguish a stakeholder from an actor.	☐	☐	☐
I can write an atomic functional requirement.	☐	☐	☐

I can make a non-functional requirement measurable.	☐	☐	☐
I can select a suitable verification method.	☐	☐	☐
I can separate rules, assumptions, constraints and risks.	☐	☐	☐
I can justify MoSCoW priorities.	☐	☐	☐
I can identify premature design decisions in requirements.	☐	☐	☐

Exit ticket

The strongest requirement I wrote is:

_____
_____

One requirement that still needs clarification is:

_____
_____

One assumption that could create project risk is:

_____
_____

One priority decision I would challenge is:

_____
_____

One question I will bring to the next session is:

_____
_____

Session 4 Laboratory from Requirements to User Stories and Use Cases

Session 4 Laboratory from Requirements to User Stories and Use Cases

Worked Example, Guided Practice and Independent Exercises

Laboratory overview

Session 3 established the business problem, stakeholders, requirements, rules, assumptions, constraints and priorities. This laboratory converts selected requirements into user stories, acceptance criteria, actors and use cases. The goal is to explain who receives value, what evidence demonstrates success and how an external actor achieves a goal through the system.

Learning outcomes

- Explain the purpose and limitations of user stories.
- Write role–goal–benefit stories.
- Write specific, observable and testable acceptance criteria.
- Use Given–When–Then for a concrete acceptance example.
- Distinguish requirements, user stories, acceptance criteria and use cases.
- Identify primary, supporting and offstage roles.
- Name use cases using actor goals.
- Define an appropriate system boundary.
- Create a basic UML use-case diagram.
- Build traceability from recognised need to use case.

Recommended working method

Stage	Action
Select	Choose a recognised requirement or stakeholder need.
Story	Identify a specific role, meaningful goal and value.
Criteria	Describe observable conditions that demonstrate success.
Actors	Identify external roles or systems that interact directly.
Use cases	Name meaningful goals within the agreed system boundary.
Diagram	Show scope and participation, not internal design or message order.
Trace	Link every artefact back to a recognised problem or requirement.
Review	Check clarity, testability, consistency and technology neutrality.

Texts supporting this laboratory

- Craig Larman, Applying UML and Patterns, 3rd edition, Chapter 6, Use Cases. The chapter treats use cases as text stories for discovering and recording requirements and covers brief, casual and fully dressed formats as well as UML use-case diagrams.
- Brett McLaughlin, Gary Pollice and David West, Head First Object-Oriented Analysis & Design, Chapter 2, Gathering Requirements, with Chapter 3 as useful follow-on reading on requirements change.

The laboratory combines Larman’s goal-oriented, text-first use-case approach with the accessible requirements and customer-value emphasis of Head First OOAD.

Concept guide

One capability, four artefacts

Artefact	Question	Example
Requirement	What capability or quality is required?	The system shall allow a member to search by title,
User story	Who needs what, and why?	As a member, I want to search the catalogue so that I can find
Acceptance criteria	What observable evidence	Title, author and ISBN are accepted; matching results
Use case	How does an actor achieve a meaningful goal	Search Catalogue

User-story pattern

```
As a <specific role>, I want <meaningful goal>, so that <stakeholder benefit>.
```

Part	Purpose	Quality question
Role	Identifies who receives value.	Is the role specific and meaningful?
Goal	States the outcome sought.	Is it a stakeholder goal rather than a screen action?
Benefit	Explains why the outcome matters.	Does the benefit expose value, risk or misunderstanding?
A story is not a full specification Do not attempt to put every validation rule, alternative path, screen layout, database structure, Java class or security condition into one story. Stories prompt conversation; supporting artefacts carry detail.		

Acceptance-criteria qualities

Quality	Meaning
Specific	Clear enough to interpret consistently.
Observabl	The result can be seen or inspected.
Testable	Evidence can determine whether the condition is
Relevant	Directly supports the story or capability.
Consistent	Matches requirements and business rules.
Technolog	Avoids needless implementation detail.

Given–When–Then

```
Given <starting context>
And <additional condition>
When <event>
Then <observable result>
And <additional result>
```

Given describes context, When describes the event, and Then describes an observable result. Do not use Given–When–Then to prescribe internal code.

Actors, use cases and system boundary

Actor categories

Category	Meaning	Example
Primary	Seeks a goal from the	Member reserves a
Supporting actor	Provides a service required by the use case.	Payment or authentication service.
Offstage stakeholder	Has interests that must be protected but no direct	Data-protection adviser or regulator.

What makes a useful use case?

- A recognisable actor goal.
- Interaction with the system under discussion.

- A meaningful outcome or value.
- A goal within the agreed system boundary.
- A concise verb–object name such as Register Participant or Cancel Booking.

Basic use-case notation

Element	Notation and meaning
Actor	External role or system, placed outside the system
Use case	Oval containing a verb–object actor goal.
System	Named rectangle containing the use cases.
Association	Simple line showing actor participation, not timing

Common mistakes

- Actors inside the boundary.
- Use cases outside the boundary.
- Buttons, fields or screens treated as use cases.
- Database tables or Java classes included.
- Message order or timing represented.
- Every actor connected to every use case.
- One huge “Manage System” use case.
- Unnecessary use of include and extend.

Keep the first diagram simple
Use actors, actor goals, a named boundary and associations. Correct basic modelling is more valuable than advanced

Part A: Worked example

Riverside Community Sports Centre

The centre supports memberships, facility bookings, fitness-class registrations and waiting lists. Members currently contact reception to check availability. Waiting lists are maintained manually, and conflicting facility requests sometimes occur.

Worked requirement

FR04: The system shall allow an active member to book an available facility for a selected date and time.

Worked user story

As an active member, I want to book an available facility for a selected date and time so that I can confirm my planned activity without contacting reception.

Story	Worked content	Reason
Role	Active member	The role is specific and reflects an eligibility rule.
Goal	Book an available facility for a selected date and time	The goal is meaningful and externally visible.
Benefit	Confirm a planned activity without contacting	The benefit connects to reduced routine enquiries

Worked acceptance criteria

- The member is authenticated.
- The member account is active.
- The selected facility is available for the requested period.
- The booking does not overlap an existing confirmed booking for the facility.
- The booking records the member, facility, date, start time, end time and status.
- A confirmation is presented to the member.
- The booking appears in the member’s future bookings.

Worked Given–When–Then example

Given the member is authenticated
And the account is active
And the facility is available for the requested period
When the member confirms the booking
Then the booking is recorded

And a confirmation is presented
And the facility is no longer shown as available for that period

Questions revealed by the criteria

- How far in advance may a facility be booked?
- What is the maximum booking duration?
- When may a booking be cancelled?
- Are recurring bookings allowed?
- Does one membership type receive priority?
- What happens if the facility becomes unavailable after confirmation?

The correct response to missing information is to record a question, not silently invent a rule.

Worked actors and use cases

Role	**Classification**	**Reason**
Member	Primary actor	Seeks booking and registration
Reception	Primary actor	Carries out operational goals on
Payment provider	Supporting actor if called	Provides an external payment service where payment is in
Centre	Primary actor	Seeks operational information
Data-protection	Offstage stakeholder	Has privacy interests but may not interact directly.

Worked use-case catalogue

Use case	**Primary actor**	**Meaningful outcome**
View Facility	Member	Suitable facilities and times
Book Facility	Member	A confirmed facility booking
Cancel Booking	Member	A future booking is cancelled according to
Register for Class	Member	A class place or waiting-list position is recorded.

Record Attendance	Instructor	Attendance is recorded for a
Generate Usage	Centre	Operational usage

Worked traceability chain

Business problem	**Requirement**	**User story**	**Acceptance evidence**	**Use case**
Members must contact reception	FR02: View facilities available for a selected	As a member, I want to view availability so that I can plan	Results show facility, time and	View Facility Availability
Conflicting requests may be accepted.	FR03: Prevent overlapping confirmed bookings.	As a member, I want reliable booking confirmation so that I can	Conflicting confirmation is rejected.	Book Facility

Diagram-planning decisions

- Place Member, Reception Staff, Instructor and Centre Manager outside the system boundary.
- Place actor-goal use cases inside the named Sports Centre Management System boundary.
- Use simple associations to show participation.
- Do not show booking classes, databases, screen navigation or message sequence.

Part B: Guided practice

Task 1: Improve weak user stories

Indicative time: 12 minutes. Purpose: Recognise vague roles, missing value and implementation detail.

Weak story	**Problem(s)**	**Improved story**
As a user, I want the system to be		

As an administrator, I want a database		
As a participant, I want notifications.		
As a manager, I want a dashboard so that I can see		

Review questions

- Is the role specific?
- Is the goal meaningful?
- Does the benefit explain value?
- Has a quality requirement been mistaken for a story?
- Has a technical solution been mistaken for a stakeholder goal?

Task 2: Write acceptance criteria

Indicative time: 15 minutes. Purpose: Turn a story into observable evidence and reveal missing information.

Story: As a sports centre member, I want to join the waiting list for a full class so that I may receive a place if one becomes available.

Criterion number	Acceptance criterion
1	
2	
3	
4	
5	
6	

Given–When–Then example

Unanswered questions revealed

Question	Why the answer matters

Task 3: Classify actors and stakeholders

Indicative time: 10 minutes. Purpose: Distinguish direct interaction from broader interest.

Role or thing	Primary	Supporting	Offstage	Not an	Reason
Sports centre					
Reception					
Fitness					
Data-protection					
Database					
Payment					
Java Booking					
Local					

Task 4: Name use cases

Indicative time: 10 minutes. Purpose: Replace interface actions and broad labels with actor goals.

Weak name	Why it is weak	Improved actor-goal
Click Book		
Enter Member		
Open Database		
Validate Loop		
Manage Sports		
Display Report		

Part C: Independent student exercise

Harbour Arts and Workshop Centre

Harbour Arts and Workshop Centre runs evening courses, weekend workshops and community exhibitions. Registrations are currently received by email, telephone and paper forms. Participant details are copied into separate spreadsheets. Staff cannot always determine whether an activity has reached capacity, and waiting lists are managed through email folders. Participants sometimes receive late or inconsistent information about cancellations and timetable changes. Tutors receive attendance lists in different formats, and management finds it difficult to produce reliable information about enrolment, attendance and room use.

Some activities have age restrictions, require prior experience or require materials to be purchased. Staff want participants to view activities, request a place and receive confirmation. Tutors need accurate participant lists and must record attendance. The manager needs activity and room-usage reports. Participant contact details and emergency information must be protected. The exact cancellation policy and waiting-list priority rules have not yet been agreed.

Supplied requirements

ID	Requirement
FR01	The system shall allow a prospective participant to view
FR02	The system shall allow an eligible participant to request a
FR03	The system shall prevent confirmed registrations from
FR04	The system shall place an eligible request on a waiting list
FR05	The system shall allow authorised staff to record
FR06	The system shall notify registered participants when an
FR07	The system shall allow the manager to generate
NFR0	Only authorised staff shall view participant emergency

Task 5: Develop user stories

Indicative time: 15 minutes. Purpose: Express stakeholder role, goal and value without adding implementation detail.

Write eight user stories. Each story must trace to a supplied requirement.

Story	Require	Role–goal–benefit user story
US01		
US02		
US03		
US04		
US05		
US06		
US07		
US08		

Story-quality review

- □ The role is specific and meaningful.
- □ The goal describes stakeholder value rather than a screen action.

- □ The benefit explains why the capability matters.
- □ The story avoids database, framework and programming detail.
- □ Acceptance criteria could be written for the story.

Task 6: Acceptance criteria

Indicative time: 18 minutes. Purpose: Specify observable evidence for important stories.

Select three stories. Write at least four criteria for each. Use Given–When–Then for one story.

Selected story 1

Story ID: _____ Story: _____

Criterio	Observable acceptance condition
1	
2	
3	
4	

Selected story 2

Story ID: _____ Story: _____

Criterio	Observable acceptance condition
1	
2	
3	
4	

Selected story 3

Story ID: _____ Story: _____

Criterio	Observable acceptance condition
1	
2	
3	
4	

Given–When–Then example

_____

_____

Task 7: Identify actors and use cases

Indicative time: 15 minutes. Purpose: Define external roles and meaningful goals within the system boundary.

Actor catalogue

Actor or stakeholder	Type	Goal or interest	Direct interaction?

Use-case catalogue

Use case	Primary actor	Meaningful outcome	Source requireme

Task 8: Create the use-case diagram

Indicative time: 20 minutes. Purpose: Summarise actors, actor goals and system scope.

Draw a basic UML use-case diagram containing a named system boundary, relevant actors, eight to ten use cases and appropriate associations. Do not use include or extend unless clearly justified.

Task 9: Build traceability

Indicative time: 10 minutes. Purpose: Demonstrate that the stories, criteria and use cases remain connected to recognised needs.

Business problem or	Requir ement	User story	Acceptance evidence	Use case

Consistency questions

- Is every use case supported by a recognised requirement or stakeholder goal?
- Does every important requirement appear in at least one story or use case?
- Do criteria reflect the supplied rules and uncertainties?
- Have offstage stakeholders been incorrectly drawn as actors?
- Does the diagram keep internal classes and databases outside the model?

Session 5 Laboratory Finding the Objects

Week 3, Session 5 Laboratory: Finding the Objects

Candidate Classes and Domain Vocabulary

Laboratory overview

The previous sessions established the business problem, stakeholders, requirements, user stories, acceptance criteria and use cases. This laboratory uses those artefacts as evidence to create a shared domain vocabulary before software design begins.

Learning outcomes

- Explain the purpose of object-oriented analysis.
- Extract candidate terms from scenario evidence.
- Distinguish conceptual classes, attributes or values, actors, operations or events, and out-of-scope terms.
- Use noun extraction critically rather than mechanically.
- Recognise transactions and historical records that deserve their own concept.
- Keep, merge, convert or remove candidates with justification.
- Write concise domain-glossary definitions.
- Trace retained concepts to recognised evidence.

Working method

Stage	**Action**
Evidence	Read the scenario, requirements, rules and use cases.
Extract	Underline or list nouns, noun phrases and important
Classify	Decide whether each term is a class, attribute, actor, event/operation or outside scope.

Test	Ask whether information, identity, relationships, lifecycle or recognised language justify the concept.
Refine	Keep, merge, convert or remove each candidate.
Define	Write a concise, domain-focused glossary.
Trace	Link each retained concept to a requirement, rule or
Review	Challenge weak evidence, synonyms and missing

Candidate-class heuristic

Question	**Evidence of a stronger conceptual class**
Information	Must the organisation remember facts about it?
Identity	Must individual instances be distinguished?
Relationship	Does it connect meaningfully to other concepts?
Lifecycle	Does it change state over time?
Language	Do stakeholders recognise and use the term

This heuristic supports discussion. It is not an automatic scoring formula. A candidate may still be retained or rejected based on the problem context.

Part A: Guided worked example

Evidence excerpt

“A member borrows an available copy of a book. The loan records the issue date and due date. If no copy is available, the member may create a reservation.”

Initial extraction

Candidate term	**Possible interpretation**
member	Role/person recognised by the domain
copy	Individual physical item
book	Publication description

loan	Borrowing transaction
issue date	Simple value describing a loan
due date	Simple value describing a loan
reservation	Request transaction with a lifecycle
borrows	Action that reveals the Loan transaction

Refined decisions

Candidate	Classificati	Decisio	Reason
Member	Conceptual class	Keep	The library retains identity, status and borrowing history.
Book	Conceptual class	Keep	Describes a publication independently of individual
BookCopy	Conceptual class	Keep	Each physical item has an independent barcode,
Loan	Conceptual class	Keep	Carries dates, status and the history of one borrowing
Reservatio	Conceptual	Keep	Has dates, status, ordering
Issue date	Attribute/	Convert	A simple value describing
Due date	Attribute/	Convert	A simple value describing
Borrow	Operation/ event	Remove from	The verb reveals the Loan concept but is not itself a

Worked glossary

Concept	Definition	Related
Member	A person registered to use the	Loan,
Book	A publication described by bibliographic information such as	Author, BookCopy
BookCopy	An individual physical item owned and made available by the library.	Book, Loan
Loan	The record of one BookCopy being issued to one Member for a period.	Member, BookCopy

Reservation	A request by a Member to obtain a Book when a copy becomes	Member, Book	
Key distinction Book, BookCopy and Loan have different identity and history: the publication description, the individual physical			

Part B: CampusHub scenario

CampusHub supports student clubs, societies and campus activities. Students currently discover activities through posters, social media and separate club mailing lists. Registration requests are received through online forms and email. Organisers maintain participant lists in spreadsheets and sometimes cannot determine whether an activity is full or whether a student has already registered.

A student may browse published activities and request a place. Each activity is organised by one club or society and may contain one or more scheduled sessions. An activity has a capacity and may have eligibility rules, such as programme, year of study or prior membership. If an activity is full, an eligible student may join a waiting list. When a confirmed participant cancels, an organiser may offer the available place to an eligible waiting student.

Organisers need an accurate list of confirmed participants and waiting students. Attendance is recorded for each scheduled session. Some activities take place in campus rooms, while others are online. The Student Engagement Manager requires summary information on registrations, attendance and activity demand. Students must receive cancellation or timetable-change information. The detailed waiting-list priority policy has not yet been agreed.

Requirements and rules

- FR01: The system shall allow a student to view published activities.
- FR02: The system shall allow an eligible student to request a place on an activity.
- FR03: The system shall prevent confirmed registrations from exceeding the activity capacity.
- FR04: The system shall record an eligible student on a waiting list when the activity is full.
- FR05: The system shall allow an organiser to record attendance for a scheduled session.
- FR06: The system shall allow an organiser to cancel an activity and notify affected students.
- FR07: The system shall allow the Student Engagement Manager to generate registration and attendance summaries.
- BR01: A student may not have more than one active registration for the same activity.
- BR02: Each scheduled session belongs to one activity.
- Q01: How should waiting-list priority be determined?

Use-case names

- View Published Activities
- Request Activity Place
- Join Waiting List
- Cancel Registration
- Record Attendance
- Cancel Activity
- Generate Participation Summary

Part C: Student activities

Task 1: Initial candidate extraction

Indicative time: 10 minutes. Purpose: Create a broad evidence-based candidate list without deciding too early.

Extract at least 20 candidate terms from the CampusHub scenario, requirements, rules and use cases. Include important verbs that may reveal events or transactions.

Candidate term	Evidence source	Initial question or observation

Continue the candidate list below if required.

_____

_____

Task 2: Classify the candidates

Indicative time: 12 minutes. Purpose: Separate domain concepts from attributes, actors, operations and implementation detail.

Candidate	Conceptual class	Attribute/value	Actor	Operation/event	Out of scope / impleme	Reason

A role can be both an actor and a conceptual class, but only retain the conceptual class if the organisation must remember information or relationships about it.

Task 3: Refine the candidate list

Indicative time: 12 minutes. Purpose: Make and justify keep, merge, convert and remove decisions.

Review all candidates and record at least twelve refinement decisions.

Candidate	Decision	Final term or destination	Evidence-based justification

Terms that require particular attention

Term	Question to consider
Student /	Synonyms, different roles or genuinely different
Club / Society	Separate concepts or one agreed term such as
Registration	Does it preserve request date, status and
Waiting list	Is the list itself a concept, or are individual WaitingListEntry records more useful?
Attendance	Temporary action or persistent
Session	Does each occurrence require date, time, status
Room	Is location structure and room capacity within
Notification	Must the organisation retain delivery status or
Summary /	Persistent domain concept or generated output?
Capacity	Class or simple attribute of Activity?
Do not force certainty If the evidence is insufficient, record the term as “requires clarification” and write the stakeholder question needed to	

Task 4: Create the domain glossary

Indicative time: 12 minutes. Purpose: Stabilise the meaning of the most important retained concepts.

Define eight retained conceptual classes. Use one clear sentence per definition and name important related concepts. Do not use Java, database or user-interface terminology.

Concept	Concise domain definition	Related	Evidence

Glossary quality check

- □ Each term is unambiguous and used consistently.
- □ Each definition is concise and domain-focused.
- □ Definitions avoid Java and database terminology.
- □ Important related concepts are named.
- □ Each concept can be traced to scenario evidence.
- □ Synonyms have been merged or explicitly distinguished.

Task 5: Trace retained concepts to evidence

Indicative time: 8 minutes. Purpose: Show that the final vocabulary is supported rather than guessed.

Retained concept	**Requirement, rule, scenario phrase or use**	**Why the concept is needed**

Suggested self-check questions

- What important information would be lost if this concept were removed?
- Does the concept have identity, history or a changing status?

- Is the term only a simple value that should describe another concept?
- Is the term an actor, and does it also need to be remembered by the domain?
- Is an important transaction or historical record missing?

Task 6: Peer review

Indicative time: 8 minutes. Purpose: Challenge weak evidence and improve the shared vocabulary.

Review prompt	Peer feedback	Revision made
Which retained concept has the		
Which removed concept may actually		
Have two terms been used for the same		
Has an attribute been promoted to a class		
Is an important event or transaction		
Does every definition use domain		

Peer reviewer

Name or initials: _____

Part D: Exit ticket and extension

Exit ticket

One candidate I retained because identity or lifecycle matters:

_____

_____

One candidate I converted to an attribute and why:

_____
_____

One synonym pair I merged or clarified:

_____
_____

One unresolved domain question that should be asked:

_____
_____

Optional extension: candidate-class scoring

Select two uncertain candidates and use the heuristic as a discussion aid. “Yes” does not automatically mean the candidate must be retained.

Candidate	Information	Identity	Relationships	Lifecycle	Recognised	Decision and rationale

Preparation for Session 6

Bring the final candidate list and glossary to Session 6. The next session will add selected attributes, meaningful associations and multiplicities to construct a UML domain model. Do not add these elements to the Session 5 submission unless instructed.

Quick reference and completion checklist

Classification guide

Classificati	Question	Examples

Conceptual class	Does the domain need to remember identity, facts,	Student, Activity, Registration
Attribute or value	Is it a simple descriptive value of another concept?	capacity, status, sessionDate
Actor	Does the external role interact with the system	Student, Organiser, Manager
Operation or event	Is it an action that may reveal a transaction	register, cancel, attend
Out of scope /	Is it a screen, database, framework, service layer or	form, table, controller

Completion checklist

- □ At least 20 candidate terms extracted.
- □ Candidates classified with reasons.
- □ At least twelve refinement decisions recorded.
- □ Synonyms and ambiguous terms reviewed.
- □ Important transactions and history considered.
- □ Eight glossary definitions completed.
- □ Eight retained concepts traced to evidence.
- □ Peer review completed.
- □ Exit ticket completed.
- □ No UML associations, multiplicities, methods or software-design components added.

Additional notes

_____
_____

Session 6 Laboratory Domain Modelling

Session 6 Laboratory Domain Modelling

Conceptual Classes, Attributes, Associations and Multiplicity

Laboratory	Details
Indicative duration	75–90 minutes
Mode	Individual modelling followed by peer review
Starting point	Session 5 CampusHub candidate list and
Main output	A reviewed UML domain model
Suggested tool	PlantUML, diagrams.net or paper for the first

Laboratory overview

Session 5 produced candidate concepts and a domain glossary. Session 6 now turns that vocabulary into a UML domain model by selecting essential attributes, adding meaningful associations and assigning multiplicities from scenario evidence.

Learning outcomes

- Explain the purpose and boundary of a domain model.
- Distinguish conceptual classes from design classes.
- Select essential attributes without introducing implementation detail.
- Add associations supported by requirements, rules or use cases.
- Name associations as readable domain sentences.
- Assign multiplicities by reading each end separately.
- Represent a transaction or relationship that carries information.
- Use generalisation only where a genuine domain “is-a” relationship exists.
- Review a domain model for common errors.

Recommended workflow

Stage	Action
Review	Confirm the retained concepts and definitions from
Select	Choose the concepts needed for the current
Describe	Add only essential domain attributes.
Connect	Add associations that must be remembered.
Name	Use short domain verb phrases where they aid
Quantify	Add multiplicity at both ends using scenario evidence.
Validate	Check transactions, history and business rules.
Review	Remove software, database and speculative detail.

Domain model versus design class diagram

Domain model	Design class diagram
Concepts from the real-world	Software classes in a proposed
Selected domain attributes	Typed fields and operations
No methods or visibility	Methods, visibility and
No navigability or software dependencies	Navigability and dependencies may be design decisions
Created during analysis	Created during object design

Part A: Guided examples

Example 1: description, physical item and transaction

Concept	Meaning	Possible attributes
Book	A publication description	ISBN, title,
BookCopy	One physical item owned by	barcode, condition,
Loan	One borrowing transaction that preserves history.	issueDate, dueDate, returnDate, status

Example 2: named associations and multiplicity

```
@startuml
class Member {
  memberNumber
}
class Loan {
  issueDate
  dueDate
}
Member "1" -- "0..*" Loan : holds
@enduml
```

- Each Loan belongs to exactly one Member.
- One Member may have zero or many Loan records.
- The solid line is a domain association, not a message or data-flow arrow.
- The label “holds” lets the relationship be read as a domain sentence.

Example 3: transaction concept

```
@startuml
class Student
class Enrollment {
  enrollmentDate
  grade
}
class Course
Student "1" -- "0..*" Enrollment : has
Enrollment "0..*" -- "1" Course : records participation in
@enduml
```

Enrollment is retained because information belongs to one Student–Course participation. This is usually clearer for beginners than specialised association-class notation.

Part B: CampusHub modelling evidence

Use the same CampusHub scenario from Session 5. The model must support activity discovery, requesting places, waiting lists, scheduled sessions, attendance, cancellations and summary reporting.

Working glossary from Session 5

Concept	Working definition
Student	A person enrolled at the institution who may request places on campus activities.
StudentOrgan	A recognised club or society responsible for
Activity	A published campus event, class or activity for which students may request places.
ActivitySessio	A scheduled occurrence of an activity with a date,
Registration	A record of one student's request or confirmed
WaitingListEn	A record that an eligible student is waiting for a
AttendanceR	A record of a registered student's attendance
Room	An identifiable campus location for activity

Relevant business rules

- A student may not have more than one active registration for the same activity.
- Confirmed registrations must not exceed the activity capacity.
- Each activity is organised by one student organisation.
- Each activity contains one or more scheduled sessions.
- Each scheduled session belongs to one activity.
- Attendance is recorded for a student at a scheduled session.
- The waiting-list priority policy remains unresolved.

Uncertainty remains visible
Do not invent a specific waiting-list ranking multiplicity or policy. Record the unresolved decision in a note or question rather than

Part C: Build the domain model

Task 1: Select essential attributes

Indicative time: 12 minutes. Purpose: Describe concepts without adding database or Java design detail.

Concept	Candidate	Keep /	Reason
Student			
StudentOrga			
Activity			
ActivitySessi			
Registration			
WaitingListE			
Attendance			
Room			

Attribute checks

- Is the value simple and meaningful to stakeholders?
- Is it needed by a requirement, rule or use case?
- Is it actually another concept with identity, history or relationships?
- Is it a foreign key or Java collection that should be replaced by an association?
- Is an operation such as isAvailable() incorrectly presented as an attribute?

Task 2: Identify and name associations

Indicative time: 15 minutes. Purpose: Add only persistent, evidence-supported domain relationships.

Class A	Association name or	Class B	Evidence	Keep ?

Association decision tests

- Must the relationship be remembered beyond one temporary calculation?
- Do stakeholders discuss, query or report on the relationship?
- Does it support a business rule or use case?
- Can the association name be read as a simple domain sentence?
- Does the association duplicate another path without adding value?

Do not add an association merely because two objects may send messages later. Interaction is a design concern; a domain association represents knowledge that must be retained.

Task 3: Assign and justify multiplicities

Indicative time: 15 minutes. Purpose: Express allowed participation as domain rules rather than typical data volumes.

Association	Multiplicity at Class A	Multiplicity at Class B	Reading from both ends	Evidence / rule

Multiplicity reference

Notation	Meaning
1	Exactly one
0..1	Zero or one
0..* or *	Zero or many
1..*	One or many
2..5	Between two and five

Reading pattern

ClassA "1" -- "0..*" ClassB

Read the value beside Class A from the perspective of one Class B instance, then read the value beside Class B from the perspective of one Class A instance.

Task 4: Validate transactions and historical records

Indicative time: 10 minutes. Purpose: Ensure important dates, statuses and outcomes have an appropriate home.

Information to retain	Best owning	Why
Date a student requested an activity		
Current registration		
Waiting-list request time and position		
Date and status of an		
Attendance status for one student at one		
Cancellation date for		
Notification delivery status, if required		

Decision prompt

Could WaitingListEntry be represented as a Registration with status WAITING? State your preferred decision and explain what information and lifecycle your model must preserve.

_____

_____

Task 5: Draw the first complete domain model

Indicative time: 20 minutes. Purpose: Combine the selected concepts, attributes, named associations and multiplicities.

Create a UML domain model with approximately six to ten conceptual classes. Include at least one transaction or historical concept. Use plain associations unless a more specialised relationship is clearly justified.

Optional PlantUML starting template

Replace the placeholders with your selected attributes, association names and multiplicities. Do not copy unsupported relationships into the final model.

```
@startuml
left to right direction
hide methods
skinparam classAttributeIconSize 0

class Student {
  studentNumber
}
```

```
class StudentOrganisation {
  organisationName
}

class Activity {
  activityCode
  title
  capacity
  status
}

class ActivitySession {
  sessionDate
  startTime
}

class Registration {
  requestDate
  status
}

class WaitingListEntry {
  requestDate
  status
}

class AttendanceRecord {
  attendanceStatus
}

class Room {
  roomNumber
}

StudentOrganisation "1" -- "0..*" Activity : organises
Activity "1" -- "1..*" ActivitySession : comprises
```

```
Student "1" -- "0..*" Registration : submits
Activity "1" -- "0..*" Registration : receives
Student "1" -- "0..*" WaitingListEntry : joins
Activity "1" -- "0..*" WaitingListEntry : has
Registration "1" -- "0..*" AttendanceRecord : has
ActivitySession "1" -- "0..*" AttendanceRecord : records
Room "1" -- "0..*" ActivitySession : hosts
@enduml
```

Treat this as a starting hypothesis
Check every attribute, association and multiplicity against the scenario. A technically valid diagram can still be a poor domain

Task 6: Peer review the model

Indicative time: 10 minutes. Purpose: Find semantic errors, unsupported detail and missing evidence.

Review question	Peer feedback	Revision made
Does every class represent a meaningful		
Are important transactions and		
Are all attributes simple descriptive values?		
Is every association supported by a		
Does every association have multiplicity at		
Can association names be read as domain		
Have methods, foreign keys and implementation		

Is any generalisation or composition genuinely		

Peer reviewer

Name or initials: _____

Part D: Diagnose a weak model

The following items have been proposed for a CampusHub domain model. Identify the problem and write a correction.

Proposed model	Problem	Correction
ActivityController		
registrationTable		
studentId inside Registration as the		
isFull() shown as an		
ArrayList<Registration> inside Activity		
Student --> Registration		
Activity 1 -- 1 Registration without		
ReportScreen		
WaitingListEntry omitted even though		
Composition diamond used for every “has”		

Part E: Exit ticket and project bridge

One multiplicity I changed after reviewing the evidence:

_____

_____

One association I removed because it was speculative:

_____

_____

One transaction or historical concept that strengthened the model:

_____

_____

One unresolved stakeholder question that affects the model:

_____

_____

Bridge to the group design project

The domain model is a useful baseline for the later group design project, but the project-management tool should not be introduced as part of today's core modelling activity. First establish a technically defensible model; then use a board to plan design work.

Recommended board	**Definition of done**
Review domain	All retained concepts have agreed
Review associations	Every relationship is supported and
Check multiplicities	Both ends are justified by rules or
Resolve modelling	Decision and rationale are recorded.
Prepare detailed use	Main, alternative and exception paths
Create Gherkin	Success, failure and boundary examples

Readings, resources and completion checklist

Core reading

- Craig Larman, Applying UML and Patterns, 3rd edition, Chapter 9: Domain Models.
- Brett McLaughlin, Gary Pollice and David West, Head First Object-Oriented Analysis & Design, Chapter 4: Analysis.

Web resources

- Project-Based Course Notes: software development lifecycle, agile methods, requirements, development and testing. https://cis-projects.github.io/project_based_course_notes/introduction.html
- Low-Level Design repository: examples and references for later object design work. https://github.com/divyadhimaan/low-level-design/blob/main/Readme.md
- Advanced Software Design materials: lecture and exercise repository for later design topics. https://github.com/kikofernandez/AdvancedSoftwareDesign

Completion checklist

- □ Essential attributes selected and justified.
- □ At least eight candidate associations considered.
- □ Final associations supported by evidence.
- □ Multiplicity shown at both ends of every final association.
- □ At least one transaction or historical concept included.
- □ Waiting-list uncertainty recorded rather than invented.
- □ Complete domain model drawn.
- □ Peer review completed and revisions recorded.
- □ No methods, visibility, navigability, controllers, repositories, screens or database structures included.

Additional notes

_____

_____

Session 7 Laboratory Detailed Use Cases

Session 7 Practical Lab Detailed Use Cases and Executable Gherkin

Why behavioural examples matter, and how selected examples become executable

Lab purpose
Continue the community equipment-loan scenario. Refine a detailed use case, distinguish the Main Success Scenario from its extensions, select important examples, express them in Gherkin, and connect the steps to JavaScript definitions in

1. Learning outcomes

- Explain why teams use detailed use cases and executable behavioural examples.
- Distinguish the Main Success Scenario from recoverable, failure and cancellation extensions.
- Select examples from use-case paths and business rules rather than translate every step.
- Write clear Given–When–Then scenarios in domain language.
- Connect Gherkin steps to simple JavaScript definitions.
- Interpret passed, failed, undefined, ambiguous and syntax-error results.

2. Why are we doing this?

The central idea General requirements can be interpreted differently by domain experts, analysts, testers and developers. A detailed use case describes the complete actor goal. Gherkin adds precise examples of important paths, outcomes and boundaries. Step

The use case itself does not become executable. Selected behaviours and rules are expressed as focused Gherkin scenarios, and those scenarios become executable through step definitions.

Artefact	**Question answered**
Evidence and	What is known, constrained or uncertain?
Detailed use case	How does the actor pursue the complete
Main Success	What is the normal successful path?
Extensions	What causes the path to depart, recover or
Gherkin example	What exactly should happen in one concrete
Executable check	Does the current implementation still behave

3. Continuing scenario

Community Equipment Loan Service The service lends equipment to registered members. Earlier labs identified Member, Equipment Type, Equipment Item, Loan Request and Loan. This lab examines the goal Submit

Known fact or rule	**ID**
Only an active borrowing account may submit a	BR-01
The equipment type must be available for the	BR-02

A standard loan may not exceed 14 days.	BR-03
A successful request is recorded as Submitted and	FR-09
The evidence does not establish a waiting-list policy.	Open

Evidence discipline
Do not invent deposits, fees, approval workflows or waiting-list

4. Flow of events: main path and extensions

Flow-of-events reminder
The flow of events describes actor-system interaction while the actor pursues the goal. It remains at the system boundary. It does not describe methods, database operations, screen

Common	Preferred module	Meaning
Main or basic flow	Main Success Scenario	Representative path in which the actor achieves the goal.
Alternative flow	Extension	A condition causes departure from a numbered main step.
Exception flow	Recoverable or failure extension	A problem or unmet rule changes the path.

An extension must state where it occurs, the condition, the response, and where the flow resumes or ends.

Type	Meaning	Example
Main Success	Goal succeeds normally.	Active account, available equipment and valid

Recoverable extension	Problem is corrected and flow resumes.	Revised dates or reduced duration.
Failure extension	Goal is not achieved.	Inactive account; no request recorded.
Cancellation	Actor abandons the goal.	Member cancels before confirmation.

Why this matters for Gherkin Success examples come from the Main Success Scenario. Failure and recovery examples come from extensions. Boundary examples come from business rules. Do not translate every use-

5. What Gherkin, Cucumber and step definitions do

Part	Purpose
Detailed use	Describes the complete goal and its paths.
Gherkin	States one concrete behavioural example.
Step	Gives a Gherkin sentence executable meaning.
Cucumber	Matches steps to definitions, runs code and
testjam.io	Teaching sandbox showing feature, definitions
VS Code	Project editor for feature files, definitions and

6. Worked example: detailed use case

6.1 Header

Field	Worked content

ID and name	UC-03 Submit Equipment Loan Request
Scope	Community Equipment Loan Service
Level	User goal
Primary actor	Member
Preconditions	The member has a borrowing account; the equipment catalogue exists.
Trigger	The member decides to request equipment for
Minimal guarantee	No invalid request is recorded; a reason is supplied when the goal cannot be completed.
Success guarantee	A valid request is recorded as Submitted and a unique reference is supplied.
Open issue	Handling beyond refusal or revised dates is not

6.2 Main Success Scenario

Step	Actor intention or system responsibility
1	The member starts a new equipment loan request.
2	The system identifies the member and borrowing account.
3	The member identifies the equipment type, quantity and
4	The system confirms that the borrowing account is active.
5	The system confirms availability for the requested period.
6	The system confirms that the duration does not exceed 14
7	The member confirms the details.
8	The system records the request with Submitted status.
9	The system provides a unique reference.

6.3 Extensions

Ref.	Condition	Response and destination
4a	Account inactive.	Explain ineligibility; record no
5a	Equipment unavailable for part	Identify unavailable period; allow revised dates; resume at step 5.
6a	Duration exceeds 14	State limit; allow revised duration;
7a	Member cancels before confirmation.	Discard unsubmitted details; end.

6.4 Quick classification check

- 4a is a failure extension.
- 5a and 6a are recoverable extensions.
- 7a is a cancellation extension.

7. From paths and rules to focused examples

Source	Concrete example	Reason
Main Success Scenario	Active account, available item, 10-day request	Representative success
Extension 4a	Inactive account	Eligibility failure
Extension 5a	Unavailable equipment	Availability
BR-03	13, 14 and 15 days	Exact threshold

```
Feature: Submit equipment loan request


  Scenario: Eligible member requests available equipment
    Given member M-104 has an active borrowing account
    And equipment type ET-12 is available for 10 days
    When the member requests ET-12 for 10 days
    Then the request is recorded with Submitted status
    And a request reference is returned
```

- Given establishes relevant domain state.
- When identifies the business event.
- Then states observable outcomes.
- The scenario avoids screens, buttons, classes and database tables.

Prediction
Without matching step definitions, Cucumber should report undefined steps. The behaviour is described, but executable

8. Practical 1: observe undefined steps

1. Open https://testjam.io/Links to an external site.

2. Locate Feature File, Step Definitions and Test Results.
3. Delete the starter text from both editor panels.
4. Paste the feature from Section 7 into Feature File.
5. Select Run Tests.
6. Record the undefined result and explain why it is useful.

Why testjam.io?
It removes installation and configuration barriers while keeping the specification, implementation glue and results visible on one screen. It is a teaching sandbox, not the normal long-term

9. Practical 2: make the success example executable

Why add step definitions?
Given definitions arrange relevant state, When definitions perform the event being examined, and Then definitions check observable results. The code below is an in-memory

```
const { Given, When, Then } = require('@cucumber/cucumber');

const assert = require('assert');


Given('member {word} has an active borrowing account',
function (memberId) {

  this.memberId = memberId;

  this.accountActive = true;

});

Given('equipment type {word} is available for {int} days', function
(type, days) {

  this.equipmentType = type;

  this.availableDays = days;

});

When('the member requests {word} for {int} days', function (type,
days) {

  this.recorded = this.accountActive && this.equipmentType ===
type &&

                this.availableDays >= days && days <= 14;

  this.status = this.recorded ? 'Submitted' : undefined;

  this.reference = this.recorded ? 'LR-1001' : undefined;

  this.refusalReason = !this.accountActive ? 'account inactive' :

    this.availableDays < days ? 'equipment unavailable' :

    days > 14 ? 'duration exceeds limit' : undefined;

});
```

1. Paste the code in Step Definitions.
2. Run and confirm a pass.
3. Change only the When value from 10 to 15 days.
4. Run and inspect the failure.
5. Explain why failed is different from undefined.
6. Restore 10 days.

10. Practical 3: add an eligibility failure

```
Scenario: Member with an inactive account is refused
  Given member M-205 has an inactive borrowing account
  And equipment type ET-12 is available for 10 days
  When the member requests ET-12 for 10 days
  Then no loan request is recorded
  And the refusal reason is account inactive
```

Run once to observe the new undefined steps, then add the minimum definitions:

```
Given('member {word} has an inactive borrowing account',
function (memberId) {
  this.memberId = memberId;
  this.accountActive = false;
});
Then('no loan request is recorded', function () {
  assert.strictEqual(this.recorded, false);
});
Then('the refusal reason is account inactive', function () {
  assert.strictEqual(this.refusalReason, 'account inactive');
```

Result	Meaning	Response
Passed	Steps matched and checks	Retain as regression
Failed	Checks contradicted expected behaviour.	Inspect code, data or expectation.
Undefine	No matching definition	Add or correct a
Ambiguo	More than one definition	Make wording or patterns
Syntax	Feature or code cannot be	Correct structure or code.

11. Practical 4: protect the 14-day boundary

```
Scenario Outline: Standard loan duration boundary
  Given member M-104 has an active borrowing account
  And equipment type ET-12 is available for 20 days
  When the member requests ET-12 for <days> days
  Then the request should be <outcome>

  Examples:
    | days | outcome  |
    | 13   | recorded |
    | 14   | recorded |
    | 15   | rejected |
```

```
Then('the request should be {word}', function (outcome) {
  const actual = this.recorded ? 'recorded' : 'rejected';
  assert.strictEqual(actual, outcome);
});
```

- Run all three rows.
- Change days <= 14 to days < 14.
- Identify why the 14-day row fails.
- Restore days <= 14.

Key insight
The 14-day example protects the exact boundary. Without it, an off-by-one error could appear correct.

12. Student task: unavailable equipment

Required behaviour
An active member requests ET-12 for 10 days, but the equipment is unavailable. No request is recorded and the result

In pairs, write one focused scenario and only the definitions needed to run it. Do not invent waiting-list behaviour.

12.1 Feature scenario

_____
_____

12.2 Step definitions

_____
_____

12.3 Review checklist

- □ Domain behaviour rather than interface detail
- □ Every sentence is necessary
- □ Exactly one definition matches each step
- □ Outcome is clear without reading JavaScript
- □ Then definition checks the promise
- □ No unsupported policy was invented

13. Why execute the examples?

Advantage	Practical value
Shared	Different roles discuss the same concrete
Earlier	Vague outcomes become observable results.
Boundary	13, 14 and 15 clarify the exact threshold.
Fast feedback	Scenarios can be rerun when code changes.
Regression	Later changes are checked against agreed
Traceability	Rules link visibly to examples and checks.

Keep the claim realistic
Passing scenarios do not prove complete requirements, security, usability, performance or defect-free software. Executable examples complement other analysis and testing activities; they

14. Moving to VS Code

testjam.io sandbox	Typical project
Feature File panel	features/equipment-loan.feature
Step Definitions panel	features/step_definitions/ equipment-loan.steps.js
Run Tests	Project test command or IDE task
In-memory demonstration	Application service, API, UI driver
Easy first experiment	Maintainable project and CI

- The official Cucumber extension adds highlighting, formatting, autocomplete and navigation.
- The extension alone does not provide a Cucumber runtime or project dependencies.

- The same scenarios can later run locally and in continuous integration.

15. Consolidation and reflection

Analysis decision	Gherkin	Developer
Success guarantee	Then checks status and reference.	Definitions inspect both outcomes.
Inactive-account extension	No record and clear reason.	Test state creates inactive account.
14-day business	Examples use 13, 14	Check protects exact
Unavailable-equipment open	No waiting-list example.	Policy must be clarified first.

What did Gherkin describe that JavaScript did not?

_____
_____

What executable meaning did JavaScript add?

_____
_____

Why are several focused scenarios stronger than one long scenario?

_____
_____

How could these examples prevent a later regression?

_____
_____

16. Applying the technique to CA1

Assessment connection
Apply the same reasoning to the allocated CA1 scenario and follow the separate CA1 specification. Do not reproduce this

Session 9 Laboratory System Sequence Diagrams

System Sequence Diagrams

From a Use-Case Scenario to Candidate System Operations

Case study Riverside Community Partnership: Community Equipment Loan Service. The lab continues the shared case

Why are we doing this?

CA1 established what the proposed system must support. The group-based CA2 begins the move from analysis toward software design. Before deciding which internal objects collaborate, the group needs an agreed view of the events that cross the system boundary. A System Sequence Diagram provides that view.

The key idea A use case describes the actor's goal and flow. An SSD extracts the actor-generated system events and observable responses. Those events become candidate system

Learning outcomes

- Explain why an SSD is used before internal design.
- Read actors, system participants, lifelines, messages, responses and alt fragments.
- Derive system events from one use-case scenario.
- Exclude internal system responsibilities from an SSD.
- Create and validate an SSD using PlantUML.
- Explain how an SSD supports the group CA2 design baseline.

1. SSD essentials

SSD definition A System Sequence Diagram is a UML sequence diagram that shows external actors interacting with the complete proposed system as one black box for one

Include	Exclude
External actor roles	Controllers and application
One system participant	Repositories and databases
Actor-generated system events	Internal checks and calculations
Boundary input parameters	Domain objects as internal
Observable responses	Buttons, forms and screens
Selected alt, opt or loop	Several unrelated actor goals

1.1 Reading sequence notation

Element	Meaning
Actor	An external role that exchanges messages with
Participant	The complete proposed system in an SSD.
Lifeline	The vertical dashed line; time moves from top
Message	An event or requested operation sent to the
Dashed return	An observable result returned to the actor.

Activation	Optional indication that the system is
alt fragment	Mutually exclusive alternative paths controlled

1.2 Prescriptive eight-step method

1. Select one use-case scenario.
2. Identify the initiating actor.
3. Classify use-case steps as actor actions or internal responsibilities.
4. Name the meaningful system events.
5. Add only the required boundary parameters.
6. Draw the successful SSD first.
7. Add selected alternatives only where useful.
8. Validate the diagram against the use case, Gherkin and domain vocabulary.

2. Worked example: Submit Standard Loan Request

Follow every stage. The aim is not merely to copy the finished diagram; the aim is to understand why each event appears and why the internal checks do not appear.

2.1 Selected scenario

Field	**Worked value**
Primary actor	Member
Goal	Submit a standard equipment-loan request.
Preconditions	The member has a borrowing account; the catalogue can be accessed.
Success guarantee	A Submitted request and unique request reference are recorded.

Selected path	Account active, equipment available and duration within 14 days.

2.2 Main Success Scenario

1. Member starts a standard loan request.
2. System identifies the borrowing account.
3. Member supplies equipment type, quantity, collection date and duration.
4. System checks the account status.
5. System checks suitable equipment availability.
6. System checks that duration does not exceed 14 days.
7. Member confirms the request.
8. System records the request.
9. System provides a reference.

2.3 Classify each step

Steps	Classification	SSD treatment
1, 3, 7	Actor actions	Possible boundary events.
2, 4, 5, 6,	Internal	Do not show as actor messages.
9	Observable system	May appear as a return

2.4 Choose the event granularity

A conversational system could use startLoanRequest(), provideRequestDetails() and confirmLoanRequest(). The worked example uses one coarse-grained event because the actor submits all information as one coherent business request.

Selected system event submitLoanRequest(memberNumber, typeCode, quantity, collectionDate, durationDays)

Selected response	requestSubmitted(requestReference)

2.5 Render the successful SSD

```
@startuml
hide footbox
actor Member
participant "Community Equipment Loan Service" as System
Member -> System : submitLoanRequest(memberNumber,
typeCode, quantity, collectionDate, durationDays)
activate System
System --> Member : requestSubmitted(requestReference)
deactivate System
@enduml
```

2.6 Explain why this is an SSD

□ Member is external to the system.

□ The complete proposed system appears as one participant.

□ The event expresses a business intention.

□ The parameters are supplied across the boundary.

□ The returned reference is observable.

□ Eligibility, availability, validation and persistence remain hidden.

2.7 Link to earlier artefacts

Earlier artefact	**SSD connection**
Detailed use case	Provides the selected scenario and event

Gherkin When	Suggests the principal actor-generated
Gherkin Then	Suggests an observable response.
Domain model	Supplies consistent names for event

3. Worked alternatives

The same submitted event may lead to different observable outcomes. An alt fragment is useful when a small number of related outcomes remain readable in one diagram.

```
@startuml
hide footbox
actor Member
participant "Community Equipment Loan Service" as System
Member -> System : submitLoanRequest(memberNumber,
typeCode, quantity, collectionDate, durationDays)
alt account inactive
    System --> Member : requestRefused("Account is not
eligible")
else equipment unavailable
    System --> Member : requestRefused("Equipment is
unavailable")
else duration exceeds 14 days
    System --> Member : requestRefused("Maximum duration is
14 days")
else request valid
```

3.1 Read the alt fragment

- Only one branch occurs for a particular request.

□ The guards describe mutually exclusive conditions.

□ The internal rules still remain hidden.

□ A refusal response includes a useful business reason.

□ Waiting-list behaviour is not shown because the policy is unresolved.

Diagram-size rule Use separate SSDs where alternatives involve many messages, additional actors or a crowded

3.2 PlantUML copy warning

Use the supplied .puml source or copy from a plain-text editor. If code copied from a formatted page contains
, > or a quoted participant name split across physical lines, replace it with plain PlantUML text before rendering.

4. Critique and repair an incorrect SSD

Incorrect diagram

```
@startuml
actor Member
participant LoanRequestController
participant MemberRepository
participant EquipmentRepository
database LoanDatabase
Member -> LoanRequestController : clickSubmitButton()
LoanRequestController -> MemberRepository :
findMember(memberId)
LoanRequestController -> EquipmentRepository :
checkAvailability(typeId)
LoanRequestController -> LoanDatabase : insertRequest()
```

4.1 Identify the problems

Problem	Why it is wrong	Correction

4.2 Expected observations

□ There is no one black-box system participant.

□ Internal design objects have been exposed.

□ clickSubmitButton() describes UI mechanics.

□ findMember(), checkAvailability() and insertRequest() are internal actions.

□ The database incorrectly communicates directly with the external actor.

4.3 Repair task

Rewrite the diagram as a valid success SSD. Use one Member actor, one complete system participant, one meaningful system event and one observable response.

_____

_____

5. Guided exercise: Record Equipment Return

This exercise continues the same case study but uses a staff actor and more than one actor-system interaction.

5.1 Scenario evidence

- Reception Staff identify an active loan using a loan reference.
- The system returns issued-item and due-date information.
- Reception Staff identify returned items and visible condition notes.
- The system records the return.
- Returned items await inspection before becoming available.
- If no active loan exists, the return cannot proceed.

5.2 Derive the interactions before drawing

Question	**Your answer**
Who initiates the scenario?	
What is the first meaningful	
What parameter does the	
What information should be	
What is the second	

Which parameters cross the	
What confirms successful	
What is the significant failure	

5.3 Suggested vocabulary

- locateActiveLoan(loanReference)
- activeLoanDetails(issuedItems, dueDate)
- recordEquipmentReturn(loanReference, returnedItems, conditionNotes)
- returnRecorded(itemsAwaitingInspection)
- loanNotFound(loanReference)

5.4 Complete the PlantUML template

```
@startuml
hide footbox
actor "Reception Staff" as Staff
participant "Community Equipment Loan Service" as System

' First event and response

' alt fragment

@enduml
```

5.5 Compare with the worked solution

```
@startuml
hide footbox
actor "Reception Staff" as Staff
participant "Community Equipment Loan Service" as System
Staff -> System : locateActiveLoan(loanReference)
alt active loan found
    System --> Staff : activeLoanDetails(issuedItems, dueDate)
    Staff -> System : recordEquipmentReturn(loanReference,
returnedItems, conditionNotes)
    activate System
    System --> Staff : returnRecorded(itemsAwaitingInspection)
    deactivate System
else no active loan found
    System --> Staff : loanNotFound(loanReference)
end
```

5.6 Explain what remains hidden

Internal action	**Why it does not appear as an actor**
Find the Loan object	
Check each item belongs to the loan	
Update returned dates	
Set item status to awaiting inspection	
Create inspection work	
Persist the return record	

6. Independent exercise: Request Loan Extension

Create an SSD using the evidence below. Do not invent payment, penalty or staff-approval rules.

- A Member identifies an active loan and proposes a new due date.
- The system checks that the Member is responsible for the loan.
- The system checks whether the new due date conflicts with an approved later reservation.
- If no conflict exists, the due date is revised and returned.
- If a conflict exists, the extension is refused with an explanation.

6.1 Event planning

Element	Your decision
Primary actor	
Selected scenario	
System event name	
Input parameters	
Success response	
Failure response	
Internal checks to	

6.2 Draw two SSDs

□ A focused success-path SSD.

□ A version using alt for approved and refused outcomes.

6.3 Plain PlantUML starting point

```
@startuml
hide footbox
actor Member
participant "Community Equipment Loan Service" as System

' Add your messages here

@enduml
```

7. SSD review checklist

- □ The title identifies one use case and scenario.
- □ The initiating actor is correct.
- □ The proposed system appears as one black-box lifeline.
- □ Every incoming message is a genuine external event.
- □ Message names express domain intention.
- □ Parameters are known at the boundary and required.
- □ Responses are observable and useful.
- □ Internal checks, objects and persistence are excluded.
- □ Alternative guards are clear and supported.
- □ The SSD agrees with the detailed use case, Gherkin and domain vocabulary.
- □ No unresolved policy has been silently invented.

□ The .puml source renders successfully.

7.1 Peer review

Strongest feature	Most important	Question for the

7.2 Revision note

Describe one change made after review and why the change improves the SSD.

_____

_____

8. Applying the technique in CA2

CA2 begins after the individual CA1 submissions. The group must not simply adopt one person's analysis without review. Group members should compare evidence, terminology, use-case flows and Gherkin examples before agreeing the baseline used for SSDs.

Portfolio progression CA1 asks what the system must support. CA2 asks which system operations are exposed, what those operations must achieve and which objects should later

8.1 Group workflow for an SSD backlog item

1. Select a significant agreed use-case scenario.
2. Confirm the actor, trigger, Main Success Scenario and selected extensions.
3. Compare related Gherkin examples and business rules.
4. Draft the SSD using agreed domain vocabulary.

5. Peer-review the black-box boundary and event granularity.
6. Commit editable .puml source through a reviewed branch or pull request.
7. Link the accepted SSD to the backlog item and the next operation-contract task.

8.2 Evidence expected from professional practice

- Editable PlantUML source, not only an image.
- A backlog card or issue identifying owner and reviewer.
- A meaningful commit and review record.
- Traceability to the selected use case and Gherkin scenarios.
- A short rationale for event granularity and major modelling decisions.

Assessment boundary Follow the separate CA2 specification for required artefacts, Scrum-light process evidence, roles and marking. The lab teaches the SSD technique and an appropriate

9. Laboratory outputs and exit check

□ Rendered successful-request SSD.

□ Rendered request-alternative SSD.

□ Corrected version of the deliberately weak SSD.

□ Record Equipment Return SSD with supported alternatives.

□ Request Loan Extension success and alternative-outcome SSDs.

□ Completed peer review and revision note.

9.1 Exit questions

1. **Why are checkEligibility() and checkAvailability() not actor-generated events?**

_____
_____

2. **Why is an SSD useful before internal object design?**

_____
_____

3. **When should a team use separate SSDs instead of one large alt fragment?**

_____
_____

4. **How does a system event become a candidate system operation?**

_____
_____

5. **What should a CA2 group reconcile before accepting an SSD?**

_____
_____

Preparation for Session 10

Bring the accepted SSDs to the next session. Session 10 will convert selected system events into operation contracts by stating noteworthy preconditions and the required postconditions in terms of conceptual objects, attributes and associations.

Final distinction SSD: what operation crosses the boundary? Operation contract: what must be true after that operation

Session 10 Laboratory Operation Contracts and System Operations

Operation Contracts and System Operations

From Behavioural Examples and SSDs to Explicit State Change

Case study Riverside Community Partnership: Community Equipment Loan Service. This laboratory continues the shared case study used in Sessions 7 and 9.

Purpose of the laboratory

This laboratory shows how to derive an operation contract from an agreed use-case scenario, Gherkin example, Domain Model and System Sequence Diagram. You will first follow one complete worked example, then complete a guided contract and finally write an independent contract suitable for use in the collaborative CA2 design process.

Learning outcomes

- Identify candidate system operations from SSD messages.
- Use Gherkin as supporting evidence when identifying preconditions, the operation and observable outcomes.
- Distinguish state conditions from processing actions.
- Write postconditions describing object creation, attribute changes, association changes and deletion.
- Use conceptual classes and associations from the Domain Model.
- Review contracts for consistency with use cases, Gherkin, SSDs and the Domain Model.

Suggested two-hour schedule

1. How this laboratory fits into the module

Requirements
↓
Use Cases and Gherkin
↓
Domain Model
↓
System Sequence Diagram
↓
Operation Contract
↓
Responsibility Assignment
↓
Design Class and Object-Interaction Diagrams

Artefact	Main question
Requirement	What capability or quality does the organisation
Detailed use	How does an actor pursue one goal?
Gherkin	What should happen in one concrete situation?
Domain	What important concepts and relationships exist?
SSD	What events and responses cross the system
Operation	What must already be true, and what conceptual

Important distinction Gherkin describes expected behaviour in concrete examples. An operation contract specifies the conceptual state before and after a system operation. The artefacts should agree, but they are not duplicates.

1.1 A useful Gherkin mapping

Gherkin	Contract contribution	Important caution
Given	Suggests relevant preconditions or	Not every Given belongs in the contract; include only noteworthy
When	Suggests the candidate system	Use the corresponding SSD event and agreed operation name.
Then	Suggests observable effects and	Translate outcomes into conceptual object and relationship changes,

Not a mechanical conversion Given does not automatically equal every precondition, and Then does not automatically contain the full conceptual state change. Use the Domain Model and business rules to complete the reasoning.

2. Operation-contract basics

An operation contract describes the state that must exist before a system operation and the conceptual state that must exist after the operation completes. The contract states what the operation achieves, not how software objects, databases or algorithms achieve it.

2.1 Recommended contract structure

Operation:

Cross References:

Preconditions:

Postconditions:

2.2 Preconditions

A precondition is a noteworthy condition that is assumed to be true before the operation begins.

Valid state statement	Invalid processing
A Member with the supplied	Find the member.
The Member account status is Active.	Check the account
An active Loan with the supplied	Search for the loan.
The EquipmentType exists.	Validate the equipment

2.3 Postconditions

A postcondition uses past-tense or completed-state wording to describe what became true.

Postcondition	Question	Example
Object creation	What new conceptual instance	A LoanRequest was created.
Attribute modificatio	Which domain value is now different?	LoanRequest.status became Submitted.
Association formation	Which conceptual objects are now	The LoanRequest was associated with the Member.
Association removal	Which earlier relationship no	The expired Reservation was disconnected from the
Object deletion	Which conceptual instance no longer	An obsolete temporary allocation was removed.

Larman-style wording Prefer statements such as “a LoanRequest was created” and “the LoanRequest was associated with the Member.” Avoid “the system creates,” “the controller calls,” or “the database saves.”

3. Complete worked example: Submit Standard Loan Request

Follow every stage. The aim is to see how the same behaviour is expressed differently across the analysis artefacts before the contract is written.

Step 1: Start with scenario evidence and a business rule

BR-01 Only a member with an active borrowing account may submit a standard loan request.

Other relevant confirmed evidence: suitable equipment must be available for the requested period, and a standard loan may not exceed 14 days.

Step 2: Review the detailed use-case flow

1. The member starts a standard loan request.
2. The system identifies the member and borrowing account.
3. The member provides the equipment type, quantity, collection date and duration.
4. The system confirms the account is active.
5. The system confirms suitable equipment is available.
6. The system confirms the duration does not exceed 14 days.
7. The member confirms the request.
8. The system records the request with Submitted status.
9. The system provides a unique request reference.

Step 3: Review the concrete Gherkin example

Scenario: Active member submits a valid request

Given member M-104 has an active borrowing account

And equipment type ET-12 has one suitable item available
When the member requests one item for 10 days
Then the request is recorded with Submitted status
And a request reference is provided

Step 4: Map Gherkin to the boundary interaction

Gherkin	Interpretation
Given active account and	Existing state relevant to eligibility
When the member requests one item for 10	Candidate system event: submitLoanRequest(...).
Then request recorded and reference returned	Observable result plus conceptual state change requiring analysis.

Step 5: Review the SSD

```
@startuml
hide footbox
actor Member
participant "Community Equipment Loan Service" as System
Member -> System : submitLoanRequest(memberNumber,
typeCode, quantity, collectionDate, durationDays)
System --> Member : requestSubmitted(requestReference)
@enduml
```

Candidate system operation submitLoanRequest(memberNumber, typeCode, quantity, collectionDate, durationDays)

Step 6: Identify noteworthy preconditions

Question	Reasoning	Decision
Does the Member	The operation uses a memberNumber and the request	Include.
Must the account	BR-01 treats active status as required existing state.	Include.
Must the Equipment Type exist?	The request line refers to a known equipment type.	Include.
Must equipment already be	Availability is checked during the submitted request. Treating it as a precondition would assume away	Do not include as a precondition
Must duration	The operation validates the supplied duration.	Do not include as a

Key lesson Do not turn every business rule into a precondition. If the operation is responsible for testing the condition and returning an alternative outcome, the condition is part of the operation behaviour rather than an assumed precondition.

Step 7: Use the Domain Model to identify state changes

Question	Domain-model evidence	Postcondition
What new object	LoanRequest is a transaction concept.	A LoanRequest was created.
What contains the requested	LoanRequest contains RequestLine.	A RequestLine was created for the requested
Which values were assigned?	LoanRequest has reference, submittedDate, collectionDate, duration and	The supplied values were assigned; status became

Which relationships were	Member submits LoanRequest; LoanRequest contains RequestLine;	The associations were formed.

Step 8: Write the completed operation contract

Operation:
submitLoanRequest(memberNumber, typeCode, quantity, collectionDate, durationDays)

Cross References:
UC-02 Submit Standard Loan Request
BR-01 Active account required
BR-02 Suitable equipment availability
BR-03 Standard duration maximum of 14 days

Preconditions:
- A Member with memberNumber exists.
- The Member account status is Active.
- An EquipmentType with typeCode exists.

Postconditions on successful submission:
- A LoanRequest was created.
- A RequestLine was created.
- The LoanRequest was associated with the Member.
- The RequestLine was associated with the LoanRequest.
- The RequestLine was associated with the EquipmentType.
- The requested quantity, collection date and duration were assigned.
- The LoanRequest status became Submitted.
- A unique request reference and submission date were assigned.

Step 9: Check the contract against every artefact

□ The use case says a valid request is recorded.

□ The Gherkin Then outcomes say Submitted status and a reference are observable.

□ The SSD operation name matches the contract operation.

□ The Domain Model supplies LoanRequest, RequestLine, Member and EquipmentType.

□ No waiting-list, deposit, database, controller or repository behaviour was invented.

Step 10: Explain what the contract still does not say

- Which software object receives the operation.
- Which class creates the LoanRequest.
- How availability is calculated.
- How the request is persisted.
- Which programming language or framework is used.

Why this matters The contract specifies the required effect while leaving responsibility assignment and internal collaboration for the next design activities.

4. Gherkin-to-contract mapping exercise

Use the failure example below to distinguish existing state, the system operation and the resulting conceptual state.

Scenario: Inactive member attempts a request

Given member M-205 has an inactive borrowing account
When the member requests equipment for 10 days

Then no request is recorded
And the member is informed that the account is not eligible

4.1 Mapping worksheet

Gherkin element	Your interpretation
Given	
When	
Then: no request is recorded	
Then: refusal reason returned	

4.2 Draft the outcome contract

Operation:

Cross References:

Preconditions:

Postconditions for the inactive-account outcome:

4.3 Worked discussion answer

Operation:
submitLoanRequest(memberNumber, typeCode, quantity, collectionDate, durationDays)

Preconditions:
- A Member with memberNumber exists.

- The Member account status is Inactive.

Postconditions for this outcome:
- No LoanRequest was created.
- No RequestLine was created.
- No reservation or loan association was formed.
- An account-ineligible outcome was returned to the actor.

Important nuance Traditional Larman operation contracts normally emphasise successful state change and noteworthy postconditions. Recording a no-change failure outcome can still be useful for teaching and traceability, but do not fill contracts with UI messages. Keep the focus on conceptual state.

5. Guided exercise: Record Equipment Return

Use the evidence and SSD below to derive the contract. Complete the worksheet before viewing the guided solution.

5.1 Scenario evidence

- Reception Staff locate an active loan using a loan reference.
- The returned physical items and visible condition are recorded.
- Each returned item enters AwaitingInspection status.
- Physical return does not make the item Available.
- The system confirms which items await inspection.

5.2 SSD interaction

@startuml
hide footbox

```
actor "Reception Staff" as Staff
participant "Community Equipment Loan Service" as System
Staff -> System : locateActiveLoan(loanReference)
System --> Staff : activeLoanDetails(issuedItems, dueDate)
Staff -> System : recordEquipmentReturn(loanReference,
returnedItems, conditionNotes)
System --> Staff : returnRecorded(itemsAwaitingInspection)
@enduml
```

5.3 Decide which SSD message needs the contract

Message	Contract required?	Reason
locateActiveLoan(loanReference)		
recordEquipmentReturn(...)		

Guidance Query operations that only retrieve information may need a simple contract or none at all. Prioritise complex or subtle operations that change conceptual state.

5.4 Preconditions worksheet

Candidate condition	Include	Reaso
An active Loan with loanReference exists.		
Each returned EquipmentItem was issued on		
Every returned item has already passed		
The Member account is Active.		

5.5 Postcondition discovery worksheet

Question	Your

What existing conceptual objects change?	
Which attributes change?	
Is a new Inspection created now, or is inspection	
Which associations are formed or retained?	
What remains deliberately unresolved?	

5.6 Write your contract

Operation:
recordEquipmentReturn(loanReference, returnedItems, conditionNotes)

Cross References:

Preconditions:

Postconditions:

5.7 Guided solution

Operation:
recordEquipmentReturn(loanReference, returnedItems, conditionNotes)

Cross References:
UC-08 Record Equipment Return
BR-05 Inspection or maintenance required before availability

Preconditions:
- An active Loan with loanReference exists.

- Each returned EquipmentItem is associated with a LoanItem in that Loan.

Postconditions:
- The returned date was assigned to each returned LoanItem.
- Each returned EquipmentItem status became AwaitingInspection.
- The supplied visible-condition notes were recorded for the return.
- The Loan return status was updated to FullReturn or PartialReturn, as appropriate.
- No returned EquipmentItem became Available as a result of this operation.

Modelling note A separate EquipmentReturn concept is defensible only if the agreed Domain Model retains return as a transaction with identity and lifecycle. The supplied in-class model records return information through LoanItem, so the guided solution does not invent a new class.

6. Independent exercise: Request Loan Extension

Complete this contract independently, then compare your reasoning with another student. Do not invent fees, penalties, automatic suspension or staff approval.

6.1 Evidence

- A Member identifies an active Loan and proposes a new due date.
- The Member must be responsible for the Loan.

- The proposed date must not conflict with an approved later Reservation for any issued item.
- If no conflict exists, the revised due date is approved.
- If a conflict exists, the extension is refused and the due date remains unchanged.

6.2 Gherkin examples

Scenario: Extension is approved
Given member M-104 is responsible for active loan L-301
And no issued item has a conflicting approved reservation
When the member requests a new due date of 18 October
Then the extension is approved
And the loan due date becomes 18 October

Scenario: Extension conflicts with a later reservation
Given member M-104 is responsible for active loan L-301
And an issued item is reserved for another approved request
When the member requests a conflicting new due date
Then the extension is refused
And the original due date remains unchanged

6.3 SSD event

requestLoanExtension(loanReference, proposedNewDueDate)

6.4 Planning worksheet

Contract element	Your decision
Preconditions	
Successful postconditions	
Refused-outcome postconditions	
Domain concepts used	

Unsupported details deliberately excluded	

6.5 Complete contract

Operation:
requestLoanExtension(loanReference, proposedNewDueDate)

Cross References:

Preconditions:

Postconditions on approval:

Postconditions on refusal:

---

7. Contract quality review

□ The operation name and parameters match the SSD.

□ Cross references identify the relevant use case, rules or Gherkin scenarios.

□ Preconditions describe existing state, not checks or actions.

□ Postconditions use past-tense completed-state wording.

□ Created objects are identified.

□ Changed attribute values are identified.

□ Associations formed or removed are identified.

□ The vocabulary comes from the agreed Domain Model.

□ No controller, repository, database, screen or algorithm appears.

□ No unsupported business rule has been introduced.

□ Failure outcomes do not accidentally create success-state objects.

□ The contract is concise enough to help responsibility assignment.

7.1 Repair weak contract statements

Weak statement	Rewrite as a state
The system checks that the member is	
The controller creates the request and	
The database is updated with the new	
The system processes the returned	
The user sees a success popup.	

7.2 Peer review

Strongest	Most important	Question for the

7.3 Revision note

Record one revision made after the review and explain why the revision improves the contract.

_____

_____

8. Bridge to CA2: using contracts in collaborative design

CA2 is the collaborative design phase. Groups should not copy one member's CA1 analysis without review. The group should reconcile the selected use case, Gherkin examples, Domain Model and SSD before agreeing operation contracts.

Group step	Evidence
1. Agree the scenario	One detailed use-case scenario and
2. Agree the	Success, failure and boundary Gherkin
3. Agree the boundary	Reviewed SSD and candidate system
4. Agree the conceptual	Refined shared Domain Model.
5. Write the contract	Preconditions and postconditions using
6. Peer review	Consistency check and recorded
7. Use in next design	Responsibilities, CRC cards and

8.1 Suggested repository location

```
docs/
  use-cases/
  operation-contracts/
    submit-loan-request.md
    record-equipment-return.md
  decisions/
diagrams/
  ssd/
  domain-model.puml
features/
```

Tool guidance Operation contracts do not need a specialist modelling tool. A structured Markdown or Word template stored with the related SSD, Gherkin and Domain Model is suitable.

Version the editable source in Git and link the work to the relevant Trello or Planner backlog item.

8.2 Definition of Done for a CA2 contract

- □ Based on an agreed SSD operation.
- □ Uses the current shared Domain Model vocabulary.
- □ Cross-referenced to the relevant use case, rules and examples.
- □ Reviewed by another group member.
- □ Review comments resolved.
- □ Editable source committed to the group repository.
- □ Linked to the next responsibility-assignment task.

Assessment note Follow the separate CA2 specification for the required number of contracts, Scrum-light evidence, contribution records and submission format. This laboratory demonstrates the technique and a reusable workflow.

9. Laboratory outputs

- □ Completed worked-example annotations.
- □ Completed inactive-account mapping exercise.
- □ Guided Record Equipment Return contract.
- □ Independent Request Loan Extension contract.

□ Completed quality-review checklist.

□ Peer-review comments and revision note.

9.1 Exit reflection

1. **Why should equipment availability not automatically be listed as a precondition of submitLoanRequest()?**

_____
_____

2. **How did the Domain Model reveal postconditions that were not stated explicitly in the Gherkin Then steps?**

_____
_____

3. **Why might a simple query operation require less contract detail than a state-changing operation?**

_____
_____

4. **How will an operation contract help the group assign responsibilities in Session 11?**

_____
_____

Final takeaway Gherkin gives a concrete behavioural example. The SSD identifies the system operation. The operation contract specifies the required conceptual state change. The next step is deciding which objects should be responsible.